

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I SK.Arshad Basha(1923123) Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SK. Arshad Basha

92
HP

CO1-AT1 (Analytical Problem)

FOR A TIC-TAC-TOE Playing agent, define its PEAS description and characterize the environment.

Introduction

Tic-Tac-Toe is a two-player strategic game played on 3x3 grid.

An Artificial Intelligence (AI) agent playing Tic-Tac-Toe must

observe the board, decide the best move, and compete against an opponent.

PEAS Description

1. Performance measure (P):-

The performance measure defines how the success of the agent is evaluated.

- * win the game.
- * prevent the opponent from winning.
- * minimize losses.
- * make legal moves only.

2. Environment (E):-

The environment consists of everything the agent interacts with

- * 3x3 Tic-Tac-Toe board
- * Opponent layer
- * Game rule
- * Empty & occupied cells.

3. Actuators(A):

Actuators perform actions

- * Place x or o in an empty square.
- * End the game after a win (or) draw.

4. Sensors(S):-

sensors collect information

- * Read the current board configuration.
- * Detect opponent's move.
- * Identify empty positions.

Algorithms used

- * minimax Algorithm.
- * Alpha-Beta Pruning.

Advantages

- * makes optimal decisions.
- * Never makes illegal moves.
- * Predicts opponent strategy.
- * can guarantee at least a draw.

Adversarial Environment

- * Two agents compete.
- * Each player has opposite.
- * one player's success causes the other's failure.

agent must predict opponent moves.

Decision-making depends on the opponent's strategy.

Conclusion

The Tic-Tac-Toe agent operates in a fully observable, deterministic, sequential, static, discrete, known and multi-agent environment. Since both players compete with opposite objectives, it is classified as an adversarial environment.

② A warehouse company wants to deploy robots to pick and transport goods to packing stations.

Introduction

Modern warehouses use autonomous robots to transport products between storage areas and packing stations. Selecting the right intelligent agent architecture is essential for safe, efficient, and reliable operation.

Types of Intelligent Agents

① Simple Reflex Agent

- * Acts only on current perception
- * Uses condition-action rules
- * No memory.

Limitation: Cannot handle changing warehouse conditions.

② Model-Based Reflex Agent

- * maintaining an internal model
- * remembers previous observations
- * Handles partially observable environments

③ Goal-Based Agent

- * works toward predefined goals
- * plans routes
- * select actions that achieve the destination

④ Utility-Based Agent

- * minimum travel time
- * Lowest energy consumption
- * Highest safety.

⑤ Learning Agent

- * Learning element
- * Critic
- * performance element
- * problem generator

Warehouse Robot Working

- 1) Receive order
- 2) Locate product
- 3) Plan shortest route
- 4) Avoid obstacles.

product

transport to packing station

Return for next task

Advantages

- * Fast
- * Use memory
- * Achieves objectives
- * Improves continuously

Disadvantages

- * No learning
- * Limited planning
- * Slow planning
- * Complex calculation

Benefits

- * Faster deliveries
- * Higher accuracy
- * Less human effort
- * Lower operational cost
- * Better safety.

conclusion

A learning agent is the most suitable architecture because it combines perception, planning, optimization, and the learning to improve warehouse productivity and safety over time.

③ consider the map of Romania (Arad to Bucharest) as a graph of cities connected by roads with distances.

Introduction

The Romania road map is a classic AI search problem where cities are represented as nodes and roads as edges. The objective is to find a path from Arad to Bucharest using search algorithms.

Breadth-First search (BFS)

Algorithm

- 1) Start from Arad
- 2) Visit all neighboring cities
- 3) Expand nodes level by level
- 4) Stop when Bucharest is reached.

BFS Expansion order

- 1) Arad
- 2) Zerind
- 3) Sibiu

fagaras

7) Rimnicu Vilcea

8) Lugoj

9) Bucharest

BFS solution path

Arad $\rightarrow$ Sibiu $\rightarrow$ Fagaras $\rightarrow$ Bucharest

Depth - first search (DFS)

Algorithm

- 1) Start at Arad
- 2) Explore one branch deeply
- 3) Backtrack when necessary
- 4) Continue until Bucharest is found.

DFS Expansion order

1) Arad

2) Zerind

3) Oradea

DFS solution path

Arad $\rightarrow$ Zerind $\rightarrow$ Oradea $\rightarrow$ Sibiu $\rightarrow$ Fagaras $\rightarrow$ Bucharest

Advantages

- * complete
- * Easy to implement
- * uses less memory.
- * may quickly find a solution

DisAdvantages

- * High memory usages
- * slower for large graphs
- * may miss the shortest path
- * Not always complete.

conclusion:

Breadth First search is complete and optimal for equal-cost path but requires more memory. Depth-First search is memory efficient and explores deep branches first, but it is not guaranteed to find the shortest path and may not be complete in infinite search spaces.